

VUE.JS

Aula 3: Componentização

Dividindo a interface em blocos reutilizáveis: componentes, props e emits

O que vamos ver hoje

1

Por que dividir em componentes?

O problema de uma aplicação inteira em um único bloco

2

Criando e usando um componente

`app.component()` e a nova tag no HTML

3

props: dados do pai para o filho

Como um componente recebe informação de fora

4

emits: eventos do filho para o pai

Como um componente avisa o pai que algo aconteceu

AULA 3 / 7

Hoje sua aplicação deixa de ser um bloco só: você vai aprender a separar a interface em peças menores que conversam entre si.

Revisão rápida da Aula 2

Antes de seguir, lembre o que já sabemos fazer:

v-on / @

Captura eventos do usuário e chama métodos

v-model

Liga um input aos dados nos dois sentidos

v-for / :key

Renderiza listas com identidade estável

computed

Recalcula só quando as dependências mudam

Hoje: paramos de escrever tudo em um único `createApp({...})` e aprendemos a dividir a tela em componentes menores e reutilizáveis.

Por que dividir em componentes?

Até agora, tudo vivia dentro de um único `createApp({...})`. Isso funciona — até a tela crescer.

Sem componentes

- ✗ Um único `app.js` com `data()`, `methods` e `template` gigantes
- ✗ O mesmo pedaço de HTML (ex: um card de tarefa) é copiado e colado em vários lugares
- ✗ Corrigir um bug em um lugar não corrige nos outros
- ✗ Fica difícil saber “onde” cada parte da tela é controlada

Com componentes

- ✓ Cada pedaço da tela vira um componente próprio, com seu `template` e seus dados
- ✓ Um componente pode ser reaproveitado quantas vezes for preciso
- ✓ Corrigir ou melhorar um componente atualiza todos os lugares onde ele é usado
- ✓ A aplicação vira uma árvore organizada de peças pequenas

Pense em um componente como uma peça de Lego: você monta a peça uma vez e encaixa ela onde quiser, quantas vezes precisar.

O que é um componente?

Um componente é como uma mini aplicação Vue: tem seu próprio template, seus próprios dados e (se precisar) seus próprios métodos.

template

O HTML que esse componente sabe renderizar

data()

Os dados que pertencem só a esse componente

methods

As ações que esse componente sabe executar

Analogia: se `createApp({ ... })` cria a aplicação inteira, um componente é uma versão menor e reutilizável dessa mesma ideia — com a diferença de que ele vive dentro de outro componente (o "pai").

Todo componente Vue é, na essência, um objeto parecido com o que já usamos em `createApp()` — só que registrado com um nome e usado como uma tag HTML.

Criando um componente

Um componente é registrado com `app.component()`, recebendo um nome e um objeto de configuração — parecido com o que passamos em `createApp()`.

```
// app.js
const { createApp } = Vue;
const app = createApp({
  data() {
    return { titulo: "Minha Lista" };
  }
});

// registrando o componente
app.component("tarefa-item", {
  template: `<li>{{ texto }}</li>`,
  props: ["texto"]
});

app.mount("#app");
```

Repare

"tarefa-item" é o nome do componente — ele vira uma tag HTML: `<tarefa-item>`

template define o HTML que esse componente vai desenhar na tela

props (próximo slide) lista o que esse componente recebe de fora

O componente precisa ser registrado **antes** do `app.mount()`

Usando o componente no HTML

Depois de registrado, o componente vira uma tag como qualquer outra do HTML — o Vue reconhece e substitui pelo template dele.

```
<!-- index.html -->
<div id="app">
  <h1>{{ titulo }}</h1>
  <ul>
    <tarafa-item texto="Estudar Vue">
    </tarafa-item>
    <tarafa-item texto="Fazer exercício">
    </tarafa-item>
  </ul>
</div>
```

O que acontece aqui

1. O Vue encontra a tag <tarafa-item>
2. Ele troca essa tag pelo template do componente
3. O atributo texto="..." entra como prop
4. Cada <tarafa-item> vira um na tela

Chamamos <tarafa-item> de componente filho, e o <div id="app"> (ou quem o contém) de componente pai. Essa relação pai/filho é a base de tudo que vem a seguir.

Componente pai e componente filho

Toda comunicação entre componentes acontece dentro dessa relação: um componente pai contém um ou mais componentes filhos.



props levam dados do pai para o filho. **emits** levam avisos (eventos) do filho de volta para o pai. É sempre uma via de mão única em cada direção — nunca o filho muda o dado do pai diretamente.

props: enviando dados do pai para o filho

props funcionam como parâmetros de uma função: o pai "passa" o valor, e o filho só recebe e usa.

```
// registrando o componente filho
app.component("tarefa-item", {
  props: ["texto", "feita"],
  template: `<li>
    {{ texto }} —
    {{ feita ? "OK" : "Pendente" }}
  </li>`
});

<!-- pai envia os valores -->
<tarefa-item
  texto="Estudar Vue"
  :feita="true">
</tarefa-item>
```

Duas formas de passar valores

texto="Estudar Vue"

Sem os dois-pontos: o valor é sempre texto (string) fixo.

:feita="true"

Com **:** o valor é interpretado como JavaScript — pode ser um boolean, número, ou uma variável do pai.

Regra de ouro: dentro do componente filho, uma prop nunca deve ser alterada diretamente (nada de `this.texto = ...`). Quem é dono do dado é sempre o pai — o filho só exibe.

Declarando e validando props

Existem duas formas de declarar props: uma simples (lista de nomes) e uma mais completa (com tipo e valor obrigatório).

Forma simples

```
props: ["texto", "feita"]
```

Boa para começar: apenas lista os nomes que o componente espera receber. Não avisa se você errar o tipo do valor.

Forma com validação (recomendada)

```
props: {  
  texto: {  
    type: String,  
    required: true  
  },  
  feita: {  
    type: Boolean,  
    default: false  
  }  
}
```

Diz o tipo esperado (String, Boolean, Number...), se é obrigatória e qual valor usar quando ela não for informada. O Vue avisa no console se algo não bater.

emits: comunicação do filho para o pai

Como o filho não pode alterar uma prop, ele avisa o pai com um evento — e é o pai quem decide o que fazer com esse aviso.

```
// componente filho
app.component("tarefa-item", {
  props: ["texto", "feita"],
  emits: ["marcar-feita"],
  template: `<li
    @click="$emit('marcar-feita')">
      {{ texto }}
    </li>`
});

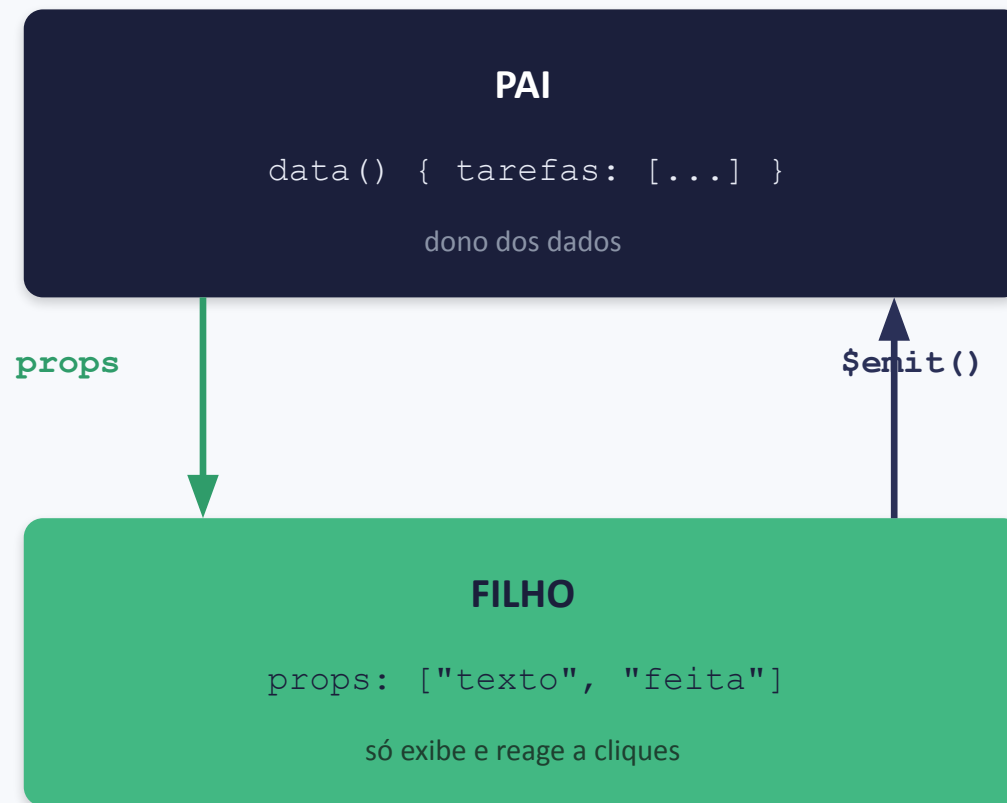
<!-- pai escuta o evento -->
<tarefa-item
  texto="Estudar Vue"
  @marcar-feita="concluirTarefa">
</tarefa-item>
```

Passo a passo

- 1 O usuário clica no dentro do componente filho
- 2 O filho executa \$emit("marcar-feita") — só isso, ele não sabe (nem precisa saber) o que vai acontecer depois
- 3 O pai, que está escutando com @marcar-feita, executa seu próprio método concluirTarefa()
- 4 É o pai quem efetivamente muda o dado (ex: feita = true)

O fluxo de dados: props para baixo, eventos para cima

Esse é o conceito mais importante da aula. Se ele ficar claro, o resto é só sintaxe.



Dados sempre descem (via props) e **eventos sempre sobem** (via \$emit). Nunca o contrário — o filho jamais escreve direto em um dado do pai.

Juntando tudo: contador com componente filho

Um botão dentro do componente filho avisa o pai, e é o pai quem guarda e atualiza o número.

```
// app.js
const app = createApp({
  data() {
    return { contador: 0 };
  },
  methods: {
    incrementar() {
      this.contador++;
    }
  }
});
app.component("botao-somar", {
  emits: ["clizou"],
  template: `<button
    @click="$emit('clizou')">
    +1</button>`
});
app.mount("#app");
```

```
<!-- index.html -->
<div id="app">
  <p>{{ contador }}</p>
  <botao-somar
    @clizou="incrementar">
  </botao-somar>
</div>
```

Sequência do clique

1. Usuário clica no botão dentro de <botao-somar>
2. O filho dispara \$emit("clizou")
3. O pai escuta com @clizou e chama incrementar()
4. contador muda no pai, e {{ contador }} atualiza na tela

Recapitulando

Componentização

Divide a interface em peças menores, reutilizáveis e organizadas

`app.component()`

Registra um componente com um nome e um objeto de configuração

`props`

Levam dados do pai para o filho — o filho só recebe e exibe

`emits` / `$emit()`

Levam avisos (eventos) do filho de volta para o pai

Fluxo de dados

Dados descem por props, eventos sobem por emits — nunca o contrário

Próxima aula: Ciclo de vida e Single File Components — saindo do CDN com Vite